

A 2D FFT Accelerator for RGB Image Processing on Zynq-7000 FPGA

Giang Vinh Huy, Nguyen Quang Nhon, Pham Dinh Phuoc, Nguyen Lam Minh Nhat, Doan Nhat Hoa

Department of Computer Engineering

Faculty of Electrical and Electronics Engineering

Ho Chi Minh City University of Technology and Education (HCMUTE)

Ho Chi Minh City, Vietnam

Abstract—Bài báo này trình bày bộ tăng tốc biến đổi Fourier nhanh hai chiều (2D FFT) cho xử lý ảnh RGB trên nền tảng Zynq-7000/PYNQ-Z1. Hệ thống áp dụng mô hình đồng thiết kế phần cứng/phần mềm, trong đó Processing System chuẩn bị dữ liệu, cấu hình và kiểm tra kết quả, còn Programmable Logic thực hiện khối tính toán FFT. Bộ tăng tốc được mô tả bằng C++/HLS, sử dụng thuật toán radix-2, bảng tra hệ số twiddle và biểu diễn fixed-point cho ảnh RGB kích thước 64×64 . IP được tích hợp với AXI4-Lite cho điều khiển và AXI master cho truy cập DDR. Với cấu hình fixed-point 28/20, kiểm chứng C/RTL đạt độ trễ 420207 chu kỳ cho mỗi frame 64×64 , sử dụng 66 BRAM_18K, 8 DSP, 9869 FF và 12377 LUT ở mức IP. Thiết kế đóng timing sau implementation với slack dương và công suất on-chip ước lượng 1.493 W.

Index Terms—2D FFT, tăng tốc FPGA, Zynq-7000, Vitis HLS, xử lý ảnh RGB, đồng thiết kế phần cứng/phần mềm.

I. GIỚI THIỆU

Biến đổi Fourier hai chiều là một công cụ quan trọng trong xử lý ảnh số vì cho phép phân tích ảnh trong miền tần số, nơi các đặc trưng như biên, chi tiết nhỏ, nhiễu và kết cấu được thể hiện rõ hơn. Tuy nhiên, phép biến đổi Fourier rời rạc hai chiều (2D DFT) có chi phí tính toán lớn. Thuật toán biến đổi Fourier nhanh (FFT) giảm đáng kể số phép toán bằng cách phân rã phép biến đổi thành các bước nhỏ có cấu trúc lặp lại [1]. Đối với ảnh hai chiều, 2D FFT thường được thực hiện theo phương pháp hàng-cột, tức là FFT một chiều được áp dụng lần lượt trên các hàng và các cột.

Trong các hệ thống nhúng, việc xử lý 2D FFT hoàn toàn bằng CPU có thể gây ra độ trễ cao, đặc biệt với ảnh RGB gồm nhiều kênh dữ liệu. PYNQ-Z1 sử dụng SoC Zynq-7000, kết hợp Processing System (PS) dựa trên ARM và Programmable Logic (PL) trên cùng một chip [2]. Kiến trúc này phù hợp cho đồng thiết kế phần cứng/phần mềm: PS đảm nhiệm chuẩn bị dữ liệu, cấu hình và kiểm tra kết quả, trong khi PL thực hiện phần tính toán FFT có khả năng song song hóa cao.

Nghiên cứu này xây dựng bộ tăng tốc 2D FFT cho ảnh RGB kích thước 64×64 trên PYNQ-Z1. IP phần cứng được mô tả bằng HLS, tích hợp vào hệ thống Zynq và giao tiếp với PS thông qua AXI4-Lite cho điều khiển và AXI master cho truy cập DDR. Hệ thống tách ảnh thành ba kênh R, G, B, thực hiện 2D FFT riêng cho từng kênh và xuất kết quả phần thực/phần ảo để so sánh với dữ liệu tham chiếu thuật toán.

Các đóng góp chính của bài báo gồm:

- Đề xuất và hiện thực một bộ tăng tốc 2D FFT cho ảnh RGB 64×64 trên Zynq-7000 bằng phương pháp đồng thiết kế phần cứng/phần mềm.
- Xây dựng datapath FFT hàng-cột radix-2 dùng fixed-point và bảng tra twiddle 64 điểm.
- Đánh giá ảnh hưởng của tối ưu twiddle LUT, cho thấy số DSP giảm từ 197 xuống 8 so với phiên bản tính sin/cos runtime.
- Khảo sát nhiều cấu hình fixed-point và chọn định dạng `ap_fixed<28, 20>` cho thiết kế đề xuất.
- Kiểm chứng thiết kế bằng tham chiếu thuật toán, mô phỏng mức C, kiểm chứng C/RTL và kết quả sau triển khai.

Phần còn lại của bài báo được tổ chức như sau. Mục II trình bày cơ sở lý thuyết, Mục III mô tả kiến trúc hệ thống, Mục IV trình bày quá trình hiện thực, Mục VI thảo luận kết quả đánh giá, và Mục VIII tổng kết các hướng phát triển tiếp theo.

II. CƠ SỞ LÝ THUYẾT VÀ CÔNG TRÌNH LIÊN QUAN

A. Biến Đổi Fourier Nhanh Hai Chiều

Với một ảnh $f(x, y)$ có kích thước $M \times N$, biến đổi Fourier rời rạc hai chiều (2D DFT) được định nghĩa như sau:

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) e^{-j2\pi(\frac{ux}{M} + \frac{vy}{N})}. \quad (1)$$

2D DFT chuyển ảnh từ miền không gian sang miền tần số, trong đó các hệ số tần số thấp biểu diễn thành phần biến thiên chậm, còn các hệ số tần số cao thể hiện biên, kết cấu và chi tiết nhỏ. Do đó, miền Fourier thường được sử dụng trong lọc ảnh, tăng cường ảnh và phân tích đặc trưng ảnh số [3]. Trong thiết kế được xét, ảnh RGB được tách thành ba kênh màu và 2D FFT được áp dụng độc lập trên từng kênh để giữ lại thông tin tần số của từng mặt phẳng màu.

Tính trực tiếp (1) đòi hỏi chi phí tính toán lớn vì mỗi hệ số đầu ra phụ thuộc vào toàn bộ mẫu đầu vào. FFT giảm đáng kể độ phức tạp bằng cách phân rã DFT thành các phép toán butterfly nhỏ hơn và khai thác tính đối xứng của nhân Fourier [1], [4]. Với dữ liệu hai chiều, phép biến đổi có thể được thực hiện theo phương pháp hàng-cột: tính 1D FFT cho từng hàng, lưu ma trận trung gian, sau đó tính 1D FFT cho từng cột. Cách tổ chức này phù hợp với thiết kế phần cứng vì dùng lặp lại cùng một nhân FFT 64 điểm, có luồng truy cập

bộ nhớ rõ ràng và thuận tiện để đối chiếu với kết quả tham chiếu phần mềm.

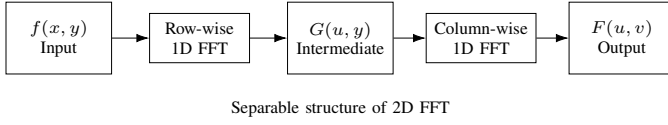


Fig. 1. Row-column decomposition of 2D FFT.

Hình 1 minh họa cách thực hiện 2D FFT bằng hai nhóm FFT một chiều. Phép biến đổi theo hàng tạo ra ma trận trung gian $G(u, y)$, sau đó phép biến đổi theo cột tạo phổ hai chiều $F(u, v)$. Nhờ tính tách biến này, phần cứng có thể tái sử dụng cùng một nhân FFT 1D thay vì hiện thực trực tiếp toàn bộ phép biến đổi hai chiều.

B. Tăng Tốc FFT Trên FPGA

FFT phù hợp với FPGA vì thuật toán có cấu trúc đều, gồm các tầng butterfly lặp lại, phép nhân với hệ số twiddle và mẫu truy cập dữ liệu có quy luật. Các đặc điểm này cho phép hiện thực đường dữ liệu chuyên biệt, dùng bộ nhớ trên chip cho dữ liệu trung gian và khai thác song song hóa tốt hơn so với bộ xử lý đa dụng [5]. Tuy nhiên, thiết kế FFT trên FPGA luôn phải cân bằng giữa thông lượng, độ trễ, tài nguyên phần cứng và độ phức tạp điều khiển.

Các kiến trúc FFT có thể ưu tiên pipeline và streaming để tăng thông lượng, hoặc tái sử dụng khối butterfly để giảm tài nguyên. Tài liệu FFT IP của AMD cũng nhấn mạnh các lựa chọn kiến trúc, scaling và xử lý tràn số khi hiện thực FFT fixed-point trên FPGA [6]. Một số nghiên cứu trước đó đã khai thác tái sử dụng tài nguyên và tổ chức bộ nhớ cho các bộ xử lý FFT 1D/2D trên FPGA [7], [8]. Trong bài báo này, hướng tiếp cận được chọn là bộ tăng tốc để kiểm chứng, dùng lại nhân radix-2 FFT 64 điểm cho các hàng, cột và ba kênh RGB trên nền tảng Zynq-7000.

C. Số Cố Định Trong Thiết Kế FPGA

Số học dấu chấm động thuận tiện khi mô phỏng phần mềm, nhưng thường tiêu tốn nhiều tài nguyên khi tổng hợp lên FPGA. Vì vậy, các thiết kế DSP trên FPGA thường dùng số học dấu chấm cố định để giảm LUT, FF, DSP và giúp dễ đạt timing hơn. Đối lại, người thiết kế phải lựa chọn hợp lý số bit phần nguyên, số bit phân thập phân, chế độ làm tròn và cơ chế xử lý tràn số.

Vitis HLS cung cấp kiểu `ap_fixed`, cho phép mô tả số fixed-point với độ rộng tùy chọn ngay trong C++ [9]. Trong FFT, lựa chọn này rất quan trọng vì giá trị trung gian và hệ số DC có thể tăng lớn sau các tầng butterfly; do đó tài liệu FFT IP của AMD xem độ rộng từ, scaling và tràn số là các yếu tố thiết kế cần quan tâm [6]. Định dạng `ap_fixed<28, 20>` được chọn thông qua quá trình quét độ rộng bit và so sánh với kết quả tham chiếu. Cách chọn này nhằm cân bằng giữa độ chính xác của hệ số FFT và tài nguyên phần cứng trên Zynq-7000.

D. Đồng Thiết Kế Trên Zynq

Zynq-7000 là nền tảng SoC tích hợp Processing System (PS) dựa trên ARM và Programmable Logic (PL) dựa trên FPGA trong cùng một chip [2]. Kiến trúc này phù hợp với các bài toán tăng tốc nhúng, trong đó PS xử lý các tác vụ điều khiển như chuẩn bị dữ liệu, cấu hình tham số và kiểm tra kết quả, còn PL đảm nhiệm các khối tính toán lặp lại và có khả năng song song hóa cao [10].

Thiết kế được xây dựng như một IP phần cứng được điều khiển bởi phần mềm. PS lưu ảnh RGB trong DDR, cấu hình IP qua AXI4-Lite và cung cấp địa chỉ bộ nhớ đầu vào/đầu ra. PL đọc dữ liệu qua AXI master, thực hiện 2D FFT cho từng kênh màu và ghi kết quả phân thực, phần ảo trở lại DDR. Cách tiếp cận memory-mapped này đơn giản, dễ kiểm chứng và phù hợp với cấu trúc giao tiếp do HLS hỗ trợ [9].

E. Vị Trí Của Công Trình

Bảng I đặt công trình này trong bối cảnh các hướng tiếp cận liên quan. Khác với các kiến trúc FFT tập trung chủ yếu vào thông lượng hoặc tối ưu pipeline tổng quát, mục tiêu của bài báo là xây dựng một kiến trúc RGB 2D FFT nhỏ gọn, có phương pháp kiểm chứng rõ ràng trên Zynq-7000.

III. KIẾN TRÚC HỆ THỐNG ĐỀ XUẤT

A. Tổng Quan Hệ Thống

Hệ thống đề xuất được tổ chức theo cấu trúc PS/PL của Zynq-7000. PS đảm nhiệm điều khiển ứng dụng, quản lý bộ đệm DDR, bảo trì cache, cấu hình bộ tăng tốc và kiểm tra kết quả. PL chứa IP 2D FFT được mô tả ở mức C++/HLS. Bộ tăng tốc đọc ảnh RGB đầu vào từ DDR, tính FFT cho từng kênh màu và ghi các ma trận phần thực/phần ảo về DDR. Kiến trúc memory-mapped được chọn vì dễ kiểm chứng và phù hợp trực tiếp với các giao tiếp AXI4-Lite và AXI master [9], [12].

Hình 2 nhấn mạnh sự phân chia giữa điều khiển và tính toán. AXI4-Lite được dùng cho các thanh ghi điều khiển bằng thông thấp, còn AXI master memory-mapped được dùng cho đọc/ghi dữ liệu đầu vào và đầu ra. PS chuẩn bị bộ đệm RGB, cấu hình IP, khởi động bộ tăng tốc và kiểm tra kết quả. PL thực hiện các phép FFT hàng-cột lặp lại.

B. Luồng Xử Lý Ảnh RGB

Ảnh đầu vào được biểu diễn thành ba ma trận 64×64 tương ứng với các kênh đỏ, xanh lá và xanh dương. Mỗi kênh được xem như một ảnh vô hướng độc lập. Bộ tăng tốc tính 2D FFT cho từng kênh và tạo hai nhóm đầu ra cho mỗi kênh: hệ số phần thực và hệ số phần ảo.

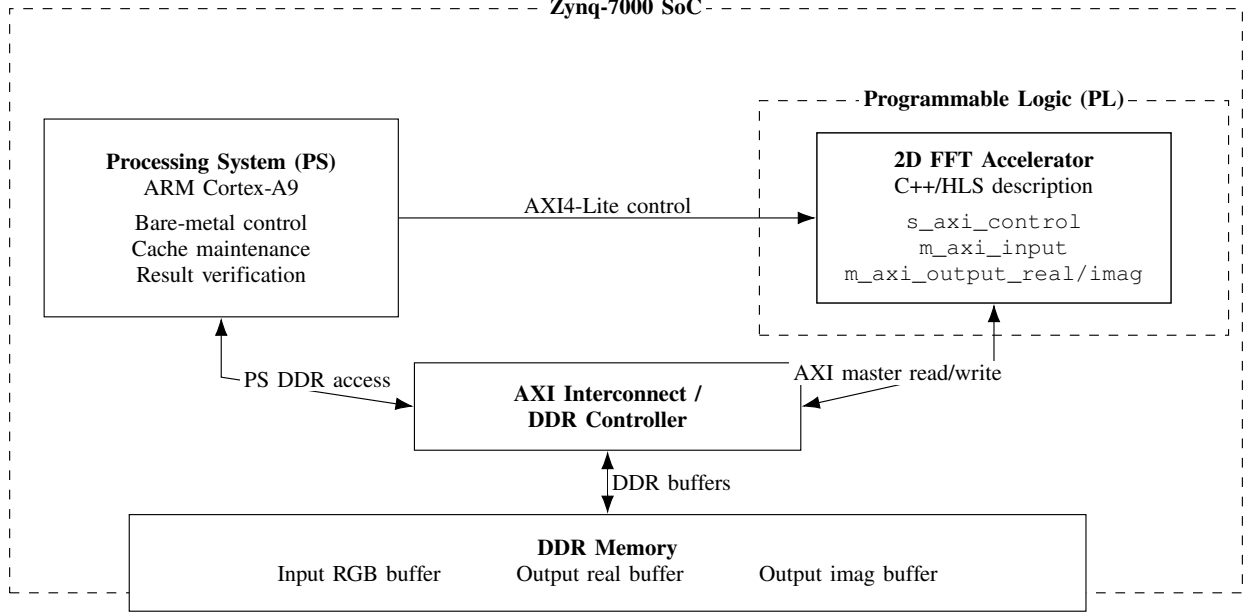
Phổ log-magnitude hữu ích để quan sát trực quan cấu trúc miền tần số, nhưng không được dùng làm tiêu chí đúng sai chính. Kiểm chứng định lượng được thực hiện bằng cách so sánh hệ số thực và ảo thô với kết quả tham chiếu phần mềm.

C. Phương Pháp FFT Hàng-Cột

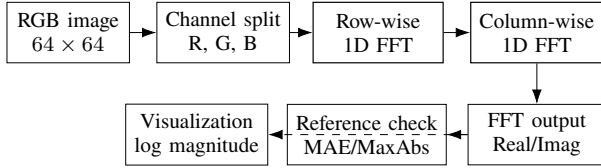
2D FFT được hiện thực bằng phân rã hàng-cột. Với mỗi kênh màu, bộ tăng tốc trước hết thực hiện FFT một chiều trên từng hàng. Kết quả trung gian được lưu trong một ma trận phức nội bộ. Sau đó, bộ tăng tốc thực hiện FFT một chiều trên từng

Bảng I
SO SÁNH VỚI MỘT SỐ HƯỚNG TIẾP CẬN LIÊN QUAN

Công trình	Nền tảng	Biến đổi	Kiểu dữ liệu	Trọng tâm
Mukherjee et al. [7]	FPGA	FFT hướng tiết kiệm diện tích	Fixed-point	Tối ưu sử dụng tài nguyên cho FFT
Mukherjee et al. [8]	FPGA	2D Fourier transform	Fixed-point	Kiến trúc 2D tiết kiệm tài nguyên
Pu et al. [11]	Zynq	Pipeline xử lý ảnh	Phụ thuộc ứng dụng	Lập trình hệ di thể cho xử lý ảnh
Công trình này	Zynq-7000	RGB 64×64 2D FFT	Fixed-point 28/20	Kiến trúc nhỏ gọn, có khảo sát bit-width và twiddle LUT



Hình 2. Kiến trúc hệ thống.



Hình 3. Luồng xử lý ảnh RGB.

cột của ma trận trung gian và ghi hệ số phần thực/phần ảo cuối cùng vào các bộ đệm đầu ra. Cách phân rã này phù hợp với tính tách biến của 2D DFT và dễ ánh xạ sang phần cứng.

Thuật toán 1. 2D FFT RGB với tính lượng giác khi chạy

Input: ảnh RGB $I[c][x][y]$, $c \in \{R, G, B\}$, $x, y \in [0, 63]$.
Output: $F_{\text{real}}[c][u][v]$ và $F_{\text{imag}}[c][u][v]$.

- Với từng kênh c , nạp ma trận đầu vào từ DDR.
- Ở mỗi tầng butterfly, tính W_N^k bằng $\sin/\cos$.
- Tính FFT 1D trên 64 hàng và lưu kết quả trung gian.
- Tính lại W_N^k , thực hiện FFT 1D trên 64 cột.
- Ghi phổ real/imag ra DDR và kiểm tra MAE/MaxAbs.

Thuật toán 2. 2D FFT RGB với bảng tra twiddle

Input: ảnh RGB $I[c][x][y]$, $c \in \{R, G, B\}$, $x, y \in [0, 63]$.
Output: $F_{\text{real}}[c][u][v]$ và $F_{\text{imag}}[c][u][v]$.

- Khởi tạo bảng twiddle cố định cho FFT 64 điểm.
- Với từng kênh c , nạp ma trận đầu vào từ DDR.
- Đọc W_N^k từ LUT và tính FFT 1D trên 64 hàng.
- Tái sử dụng LUT để tính FFT 1D trên 64 cột.
- Ghi phổ real/imag ra DDR và kiểm tra MAE/MaxAbs.

Từ góc nhìn thuật toán, cả Thuật toán 1 và Thuật toán 2 đều

dùng phân rã hàng-cột, giảm độ phức tạp từ 2D DFT trực tiếp $O(N^4)$ xuống khoảng $O(2N^2 \log_2 N)$ cho mỗi kênh $N \times N$. Khác biệt chính nằm ở cách sinh hệ số twiddle: phiên bản lượng giác tính $\sin/\cos$ trong datapath, còn phiên bản bảng tra dùng hệ số cố định và tái sử dụng cho cả hai pha hàng/cột.

IV. HIỆN THỰC PHẦN CỨNG/PHẦN MỀM

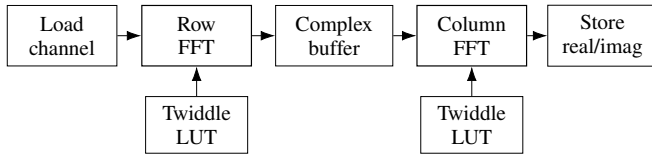
A. Kiến Trúc Bộ Tăng Tốc

Giao diện mức cao của bộ tăng tốc được mô tả như sau:

```
void fft2d_fixed_top(
    data_t input[3][64][64],
    data_t output_real[3][64][64],
    data_t output_imag[3][64][64]);
```

Trong mô phỏng tham chiếu, kiểu dữ liệu có thể được biểu diễn bằng số thực độ chính xác cao. Khi tổng hợp phần cứng, thiết kế sử dụng `ap_fixed<28,20, AP_RND, AP_SAT>` làm kiểu số cố định được chọn. Tổng độ rộng 28 bit cung cấp độ chính xác phần thập phân, trong khi 20 bit phần nguyên cung cấp dải biểu diễn đủ lớn cho các giá trị trung gian quan sát được trong các phép thử FFT 64×64 . Thiết kế dùng kích thước ảnh và số kênh tĩnh, nên cấu trúc vòng lặp và footprint bộ nhớ có tính xác định.

Hình 4 trình bày các khối chính của datapath. Điểm hiện thực quan trọng nhất là thay thế tính toán lượng giác runtime



Hình 4. Đường dữ liệu 2D FFT.

bằng bảng tra twiddle 64 điểm. Cách này tránh việc tổng hợp trực tiếp các phép $\sin$ và $\cos$ tốn tài nguyên trong datapath, nhờ đó giảm mạnh số DSP, LUT và FF.

B. Phân Chia Phần Cứng/Phần Mềm

Bảng II tóm tắt cách phân chia phần cứng/phần mềm. Phía phần mềm xử lý các tác vụ điều khiển và kiểm chứng linh hoạt, còn phần cứng thực hiện phần tính toán FFT lặp lại.

Bảng II
PHÂN CHIA PHẦN CỨNG/PHẦN MỀM

Thành phần	Nhiệm vụ
PS software	Chuẩn bị dữ liệu ảnh, quản lý bộ đệm DDR, cấu hình thanh ghi accelerator, flush/invalidate cache, khởi động IP, polling trạng thái hoàn tất, so sánh kết quả với dữ liệu tham chiếu.
PL hardware	Đọc ma trận đầu vào, tính FFT fixed-point theo hàng và cột cho các kênh R/G/B, áp dụng hệ số twiddle, ghi đầu ra phần thực và phần ảo.

C. Tổ Chức Bộ Nhớ Và Giao Tiếp AXI

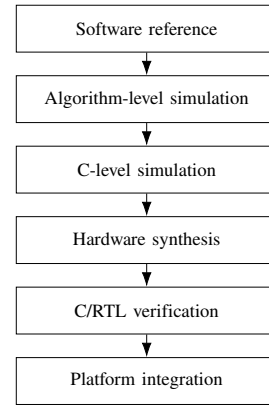
Kiến trúc được đánh giá sử dụng DDR cho các bộ đệm đầu vào và đầu ra. Bộ đệm đầu vào lưu dữ liệu ảnh RGB, trong khi hai vùng đầu ra lưu hệ số FFT phần thực và phần ảo. Các thanh ghi AXI4-Lite cung cấp thông tin điều khiển và địa chỉ bộ đệm. Các cổng AXI master memory-mapped cho phép accelerator đọc mảng đầu vào và ghi các mảng đầu ra. Cách tổ chức này ưu tiên sự đơn giản trong kiểm chứng; các phiên bản sau có thể bổ sung AXI DMA hoặc AXI Stream để tăng thông lượng.

D. Luồng Kiểm Chứng

Thiết kế sử dụng luồng kiểm chứng nhiều tầng như Hình 5. Kết quả FFT tham chiếu được sinh bằng thư viện phần mềm độ chính xác cao. Mô tả C++ được kiểm tra trước ở mức thuật toán, sau đó được đánh giá qua mô phỏng mức C, tổng hợp phần cứng và kiểm chứng C/RTL. Cuối cùng, hệ thống được tích hợp ở mức nền tảng Zynq để kiểm tra khả năng triển khai.

V. THIẾT LẬP THỰC NGHIỆM

Thực nghiệm hướng đến bo mạch PYNQ-Z1 dùng chip Zynq-7000 xc7z020clg400-1. Bộ tăng tốc được tổng hợp với ràng buộc clock 10 ns. Kích thước đầu vào cố định là ảnh RGB 64×64 . Bốn mẫu tổng hợp gồm square, gradient, checker và circle được dùng để kiểm chứng định lượng. Kết quả FFT phần mềm độ chính xác cao được dùng làm tham chiếu. Sai số được đo trên hệ số FFT thô bằng MAE và MaxAbs cho cả phần thực và phần ảo.



Hình 5. Luồng kiểm chứng thiết kế.

Bảng III
CẤU HÌNH THỰC NGHIỆM

Mục	Cấu hình
Bo mạch	PYNQ-Z1
FPGA part	xc7z020clg400-1
Công cụ thiết kế	AMD Vitis HLS / Vivado / Vitis 2025.2
Clock mục tiêu	10 ns
Clock ước lượng	7.300 ns
Kích thước đầu vào	RGB 64×64
Kiểu dữ liệu chọn	ap_fixed<28,20>
Mẫu kiểm thử	square, gradient, checker, circle
Tham chiếu	FFT phần mềm độ chính xác cao
Tiêu chí PASS	MaxAbs phần thực và ảo ≤ 1024
Thước đo	Latency, MAE, MaxAbs, BRAM, DSP, FF, LUT

VI. KẾT QUẢ VÀ ĐÁNH GIÁ

A. Mức Sử Dụng Tài Nguyên

Tài nguyên phần cứng là thước đo trung tâm của thiết kế vì mục tiêu là triển khai trên FPGA Zynq-7000 có tài nguyên hữu hạn. Bảng IV trình bày ước lượng tài nguyên ở mức IP sau tổng hợp cho cấu hình được chọn. Cột tỷ lệ dùng tổng tài nguyên của xc7z020clg400-1 làm mốc tham chiếu, nhưng phạm vi số liệu “Dùng” là riêng IP FFT chứ không phải toàn bộ hệ thống tích hợp.

Bảng IV
MỨC SỬ DỤNG TÀI NGUYÊN CỦA IP FFT

Tài nguyên	Dùng	Tổng	Tỷ lệ
BRAM_18K	66	280	23.6%
DSP	8	220	3.6%
FF	9869	106400	9.3%
LUT	12377	53200	23.3%

DSP chỉ sử dụng 3.6%, cho thấy thiết kế còn dư địa để mở rộng song song hoặc bổ sung các khối xử lý ảnh khác. BRAM_18K và LUT là hai nhóm tài nguyên có tỷ lệ sử dụng cao nhất, lần lượt ở mức 23.6% và 23.3%, nhưng vẫn nằm trong khả năng của chip mục tiêu. Số liệu ở mức IP và số liệu sau tích hợp hệ thống có phạm vi khác nhau: mức IP phần ảnh riêng khối FFT, trong khi hệ thống tích hợp bao gồm PS, AXI interconnect, reset và các logic phụ trợ. Vì vậy, các phạm vi này được dùng cho các mục đích đánh giá khác nhau và không nên so sánh trực tiếp từng dòng. Báo cáo utilization

sau implementation được trình bày cùng nhóm báo cáo timing và power ở phần sau triển khai để giữ đúng phạm vi đánh giá hệ thống.

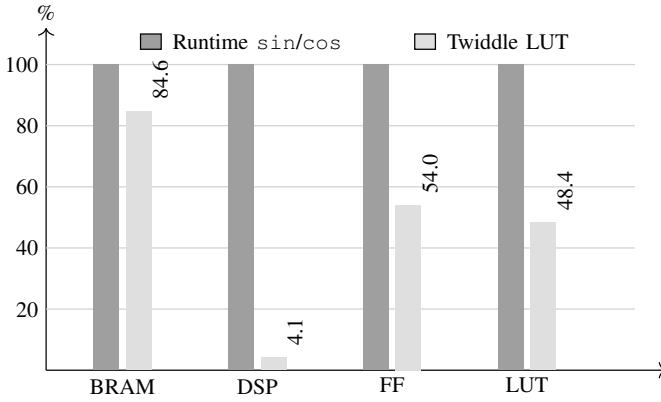
B. Ảnh Hưởng Của Tối Ưu Twiddle LUT

Mức giảm tài nguyên lớn nhất đến từ việc thay thế tính $\sin/\cos$ runtime bằng bảng tra twiddle 64 điểm. Bảng V so sánh phiên bản dùng lượng giác runtime và phiên bản dùng Twiddle LUT.

Bảng V
SO SÁNH TÀI NGUYÊN BẢNG TRA TWIDDLE

Phiên bản	BRAM	DSP	FF	LUT
Runtime $\sin/\cos$	78	197	18271	25595
Twiddle LUT	66	8	9869	12377
Mức giảm	15.4%	95.9%	46.0%	51.6%

Số DSP giảm từ 197 xuống 8, tương ứng giảm xấp xỉ 95.9%. FF và LUT cũng giảm lần lượt 46.0% và 51.6%. Kết quả này cho thấy các hàm lượng giác trực tiếp có thể chiếm chi phí lớn trong datapath FFT khi tổng hợp lên FPGA, và bảng tra là lựa chọn phù hợp cho phép biến đổi có kích thước cố định.



Hình 6. Tỷ lệ tài nguyên sau tối ưu bảng tra twiddle.

Hình 6 chuẩn hóa tài nguyên của phiên bản lượng giác runtime thành 100% để làm rõ mức thay đổi của từng nhóm tài nguyên. Sau tối ưu, DSP chỉ còn khoảng 4.1% so với phiên bản ban đầu, trong khi FF và LUT còn lần lượt khoảng 54.0% và 48.4%. BRAM giảm ít hơn vì bảng twiddle vẫn cần bộ nhớ lưu hệ số, nhưng mức giảm DSP mới là yếu tố quyết định đối với khả năng triển khai trên FPGA nhỏ.

MODULES & LOOPS	ISSUE	BRAM	DSP	FF	LUT
fft2d_fixed_top (6)		78	197	18,271	25,595
fft2d_fixed_top_Pipeline_1 (1)		0	0	8	55
fft2d_fixed_top_Pipeline_2 (1)		0	0	8	55
fft2d_fixed_top_Pipeline_VITIS_LOOP_119_1_VITIS_LOOP_120_2_VITIS_LOOP_121_3 (1)		0	0	94	283
fft2d_fixed_top_Outline_VITIS_LOOP_128_4 (1)		0	0	1,253	2,408
fft2d_fixed_top_Pipeline_VITIS_LOOP_156_12_VITIS_LOOP_157_13_VITIS_LOOP_158_14 (1)		0	0	125	294
VITIS_LOOP_142_8 (1)		0	0		

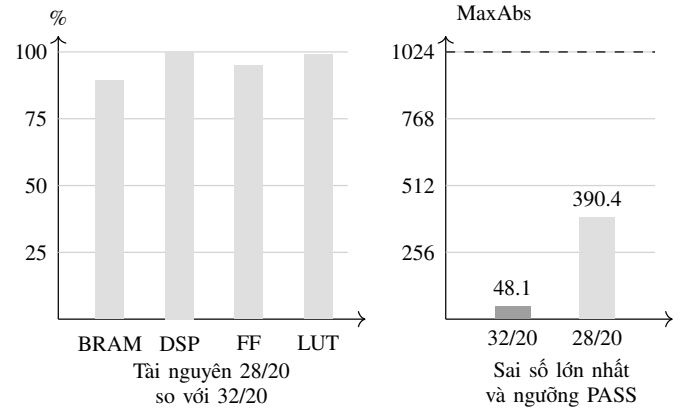
Hình 7. Báo cáo HLS của phiên bản lượng giác.

MODULES & LOOPS	ISSUE	BRAM	DSP	FF	LUT
fft2d_fixed_top (6)		66	8	9,869	12,377
fft2d_fixed_top_Pipeline_1 (1)		0	0	8	55
fft2d_fixed_top_Pipeline_2 (1)		0	0	8	55
fft2d_fixed_top_Pipeline_VITIS_LOOP_180_1_VITIS_LOOP_181_2_VITIS_LOOP_182_3 (1)		0	0	94	283
fft2d_fixed_top_Outline_VITIS_LOOP_189_4 (1)		0	0	3,296	4,048
fft2d_fixed_top_Pipeline_VITIS_LOOP_207_12_VITIS_LOOP_218_13_VITIS_LOOP_219_14 (1)		0	0	125	294
VITIS_LOOP_203_8 (1)		0	0		

Hình 8. Báo cáo HLS của phiên bản bảng tra twiddle.

C. Đánh Giá Độ Rộng Bit Fixed-Point

Nhiều cấu hình fixed-point được đánh giá để tìm điểm cân bằng giữa độ chính xác và tài nguyên FPGA. Bảng VI tóm tắt quá trình khảo sát bit-width trên bốn mẫu RGB. Một mẫu được xem là PASS khi sai số MaxAbs của cả phần thực và phần ảo không vượt quá 1024 so với tham chiếu phần mềm. Các cấu hình có số bit phần nguyên không đủ đều không vượt qua tiêu chí này vì FFT 64×64 có thể tạo ra giá trị trung gian và đầu ra lớn.



Hình 9. Đánh đổi độ rộng bit fixed-point.

Cấu hình $ap_fixed<28,20>$ được chọn vì vượt qua cả bốn mẫu kiểm thử trong khi dùng ít bit hơn $ap_fixed<32,20>$. Mặc dù định dạng 32/20 cho sai số thấp hơn, định dạng 28/20 giảm BRAM, FF và LUT trong quá trình khảo sát. Hình 9 làm rõ đánh đổi này: tài nguyên của 28/20 thấp hơn hoặc xấp xỉ 32/20, trong khi sai số MaxAbs lớn nhất vẫn thấp hơn đáng kể so với ngưỡng PASS 1024. Do đó, lựa chọn 28/20 không nhằm tối thiểu hóa sai số tuyệt đối, mà nhằm đạt cấu hình đủ chính xác với chi phí phần cứng thấp hơn.

Các số liệu tài nguyên của cấu hình 28/20 trong Bảng VI được lấy từ cùng báo cáo tổng hợp trong Vitis IDE với Hình 8, nhờ đó giữ nhất quán giữa khảo sát bit-width và đánh giá tối ưu twiddle. Các cấu hình thất bại cho thấy dải biểu diễn phần nguyên quan trọng hơn việc chỉ tăng độ chính xác phần thập phân đối với kích thước FFT được đánh giá.

D. Độ Chính Xác Và Độ Trễ

Bảng VII trình bày kết quả kiểm chứng C/RTL cho cấu hình fixed-point được chọn. Tất cả các mẫu đều vượt qua kiểm chứng so với tham chiếu phần mềm. Các giá trị MAE và MaxAbs trong bảng được tổng hợp trên cả ba kênh R, G và B.

Bảng VI
KHẢO SÁT ĐỘ RỘNG BIT FIXED-POINT

Định dạng	Pass	Worst Real	Worst Imag	BRAM	DSP	FF	LUT	Trạng thái
ap_fixed<32,20>	4/4	48.1013	34.5814	74	8	10387	12479	PASS
ap_fixed<28,20>	4/4	390.4006	308.7507	66	8	9869	12377	PASS
ap_fixed<24,20>	0/4	4928.0017	3312.7220	—	—	—	—	FAIL
ap_fixed<22,20>	0/4	44172.9983	30855.2780	—	—	—	—	FAIL
ap_fixed<24,16>	0/4	489472.0039	295416.7260	—	—	—	—	FAIL
ap_fixed<24,12>	0/4	520192.0002	326136.7223	—	—	—	—	FAIL
ap_fixed<20,12>	0/4	520192.0039	326136.7260	—	—	—	—	FAIL

Bảng VII
ĐỘ CHÍNH XÁC VÀ ĐỘ TRỄ RTL

Mẫu	Latency	Real MAE	Real MaxAbs	Imag MAE	Imag MaxAbs	Trạng thái
square	420207	2.3940	256.3373	1.6828	91.8750	PASS
gradient	420207	0.3415	93.2577	0.5043	202.1839	PASS
checker	420207	1.5384	390.4006	0.6143	308.7507	PASS
circle	420207	4.3268	240.1185	3.0387	87.1992	PASS

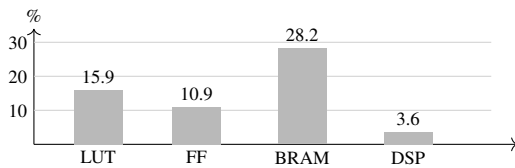
Độ trễ RTL cố định ở mức 420207 chu kỳ cho cả bốn mẫu vì kích thước đầu vào và giới hạn vòng lặp là cố định. Ở tần số 100 MHz, độ trễ này tương đương khoảng 4.2 ms cho mỗi frame 64×64 trong cấu hình memory-mapped hiện tại. Các giá trị MAE nhỏ so với dải động của hệ số FFT thô, cho thấy cấu hình fixed-point được chọn vẫn giữ được hành vi số học mong đợi trên tập kiểm thử hiện tại.

E. Tài Nguyên, Timing Và Công Suất Sau Triển Khai

Ngoài tài nguyên logic, timing closure và công suất là ba yếu tố quan trọng để đánh giá khả năng triển khai thực tế của thiết kế [13], [14]. Các báo cáo sau triển khai được tách thành từng hình riêng để giữ độ rõ của ảnh chụp từ công cụ và để mỗi nhóm chỉ số được phân tích theo đúng phạm vi của nó.

Name	Slice LUTs (53200)	Slice Registers (106400)	F7 Muxes (26600)	Slice (13300)	LUT as Logic (53200)	LUT as Memory (17400)	Block RAM Tile (140)	DSPs (220)	Bonded IOPADs (130)	BUFCTRL (32)	SCANEN2 (4)
design_1_wrapper	8461	11552	5	3735	7790	671	39.5	8	130	2	1
design_1 design_1	8013	10811	5	3504	7366	647	39.5	8	0	1	0
dbg_hub (dbg_hub)	448	741	0	245	424	24	0	0	0	1	1

Hình 10. Báo cáo sử dụng tài nguyên sau triển khai.



Hình 11. Tỷ lệ sử dụng tài nguyên sau triển khai.

Hình 10 cho thấy hệ thống sau triển khai sử dụng 8461 LUT, 11552 FF, 39.5 block RAM tile và 8 DSP. Khi quy đổi theo tài nguyên của xc7z020c1g400-1, Hình 11 cho thấy BRAM là nhóm chiếm tỷ lệ cao nhất, khoảng 28.2%, tiếp theo là LUT khoảng 15.9% và FF khoảng 10.9%. DSP chỉ ở mức 3.6%, phù hợp với kết quả HLS ở phần trước: tối ưu Twiddle LUT đã loại bỏ phần lớn nhu cầu DSP của datapath FFT. Do đó, giới hạn mở rộng của thiết kế hiện tại nhiều khả năng sẽ đến từ bộ nhớ on-chip và logic điều khiển hơn là từ tài nguyên nhân DSP.

Design Timing Summary		
Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 0.668 ns	Worst Hold Slack (WHS): 0.012 ns	Worst Pulse Width Slack (WPWS): 3.750 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 27708	Total Number of Endpoints: 27692	Total Number of Endpoints: 12659

All user specified timing constraints are met.

Hình 12. Báo cáo timing sau triển khai.

Bảng VIII
TÓM TẮT TIMING SAU TRIỂN KHAI

Chỉ số	Giá trị	Ý nghĩa
WNS	0.668 ns	Slack setup dương
WHS	0.012 ns	Slack hold dương
Endpoint thất bại	0	Đạt ràng buộc timing
Ràng buộc clock	10 ns	Mục tiêu 100 MHz
Trạng thái timing	PASS	Không vi phạm STA

Kết quả timing trong Bảng VIII cho thấy tất cả ràng buộc thời gian đều được đáp ứng sau implementation. WNS dương 0.668 ns chứng tỏ đường setup tối hạn vẫn còn biên an toàn ở chu kỳ clock 10 ns, trong khi WHS dương 0.012 ns cho thấy không xuất hiện vi phạm hold. Việc không có endpoint thất bại làm rõ rằng thiết kế có thể đóng timing ở cấu hình hiện tại, thay vì chỉ dừng ở mức mô phỏng hoặc tổng hợp HLS.

Summary

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power: 1.493 W

Design Power Budget: Not Specified

Process: typical

Power Budget Margin: N/A

Junction Temperature: 42.2 °C

Thermal Margin: 42.8 °C (3.6 W)

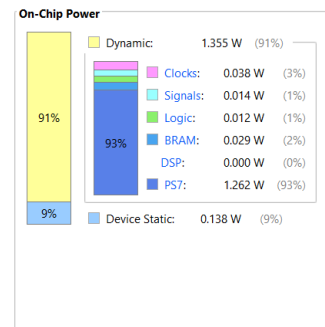
Ambient Temperature: 25.0 °C

Effective θ_{JA} : 11.5 °C/W

Power supplied to off-chip devices: 0 W

Confidence level: Medium

[Launch Power Constraint Advisor](#) to find and fix invalid switching activity

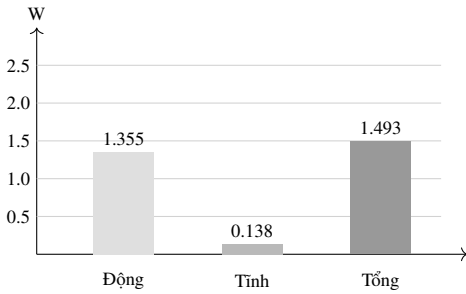


Hình 13. Báo cáo công suất sau triển khai.

Bảng IX
TÓM TẮT CÔNG SUẤT SAU TRIỂN KHAI

Thành phần	Giá trị	Ghi chú
Công suất động	1.355 W	Ước lượng sau triển khai
Công suất tĩnh	0.138 W	Ước lượng thiết bị
Tổng on-chip	1.493 W	100%
Động / tổng	90.8%	Thành phần chi phối
Tĩnh / tổng	9.2%	Nền công suất thiết bị

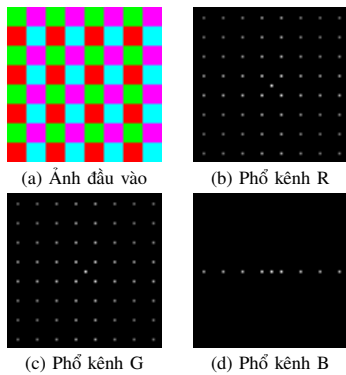
Về công suất, Bảng IX cho thấy ước lượng tổng công suất on-chip là 1.493 W, trong đó phần động chiếm 1.355 W và phần tĩnh của thiết bị chiếm 0.138 W. Công suất động tương đương khoảng 90.8% tổng công suất, phản ánh ảnh hưởng của hoạt động chuyển mạch trong hệ thống tích hợp. Một điểm đáng chú ý là PS7 chiếm tỷ trọng lớn trong công suất động, trong khi logic, BRAM và DSP của phần PL chỉ chiếm tỷ lệ nhỏ. Kết quả này phù hợp với mục tiêu của thiết kế: IP FFT trong PL tiết kiệm DSP và không tạo ra áp lực công suất lớn so với phần hệ thống xung quanh.



Hình 14. So sánh công suất on-chip.

Hình 14 trực quan hóa quan hệ giữa công suất động, công suất tĩnh và tổng công suất on-chip. Biểu đồ này không thay thế báo cáo power gốc ở Hình 13, mà chỉ chuẩn hóa các giá trị chính để thuận tiện so sánh trong phần phân tích. Do kết quả power được suy ra từ ràng buộc và activity mặc định, các giá trị này nên được xem là ước lượng sau triển khai; đo công suất thực tế trên bo mạch sẽ là bước cần bổ sung nếu muốn đánh giá năng lượng chính xác hơn.

F. Kiểm Chứng Trực Quan



Hình 15. Phổ FFT trực quan.

Bên cạnh so sánh định lượng, phổ trực quan giúp xác nhận phép biến đổi tạo ra cấu trúc miền tần số hợp lý. Hình 15

minh họa ảnh checker tổng hợp và phổ log-magnitude tham chiếu cho các kênh R, G, B sau khi dịch tần số.

Phản hiển thị trực quan không được dùng làm tiêu chí đúng sai chính vì các bước lấy magnitude, lấy log và chuẩn hóa ảnh có thể che khuất sai số ở mức hệ số. Do đó, kết quả định lượng vẫn dựa trên hệ số thực và ảo của FFT.

VII. THẢO LUẬN

Kết quả thực nghiệm cho thấy bộ tăng tốc đề xuất đáp ứng đúng chức năng và tiết kiệm tài nguyên cho xử lý 2D FFT RGB trên Zynq-7000. Tối ưu Twiddle LUT là cải tiến quan trọng nhất, giảm số DSP khoảng 95.9% đồng thời giảm đáng kể FF và LUT. Kết quả này xác nhận rằng các hàm toán học runtime cần được xem xét cẩn thận khi nhắm đến FPGA tài nguyên nhỏ.

Khảo sát fixed-point cho thấy số bit phần nguyên là yếu tố quyết định: các định dạng có ít bit phần nguyên thất bại vì FFT có thể sinh ra giá trị trung gian lớn, trong khi $ap_fixed<28,20>$ vẫn đạt kiểm chứng trên bốn mẫu và tiết kiệm tài nguyên hơn định dạng rộng hơn. Kết quả sau triển khai cũng xác nhận tính khả thi ở mức nền tảng: thiết kế đóng timing với slack dương, IP FFT chỉ dùng 8 DSP và công suất on-chip được ước lượng là 1.493 W. Tuy nhiên, cần phân biệt số liệu IP riêng khối FFT với số liệu hệ thống tích hợp gồm PS, interconnect, reset và debug logic.

VIII. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Bài báo đã trình bày một bộ tăng tốc 2D FFT cho xử lý ảnh RGB trên FPGA Zynq-7000, sử dụng FFT radix-2 hàng-cột, fixed-point và bảng tra twiddle 64 điểm. Cấu hình được chọn đạt độ trễ RTL 420207 chu kỳ cho mỗi frame 64×64 và sử dụng 66 BRAM_18K, 8 DSP, 9869 FF và 12377 LUT ở mức IP. Trong tương lai, hệ thống có thể mở rộng sang AXI DMA/AXI Stream, xử lý song song các kênh RGB, ảnh lớn hơn và pipeline video thời gian thực.

MÃ NGUỒN VÀ VIDEO MINH HỌA

Mã nguồn và tài liệu tái lập được cung cấp tại <https://github.com/vinhhuyl5/zynq-2d-fft-project-2d-fft-vivado>. Video demo có tại <https://youtu.be/MT-Wih9r9Ww?si=f5bfxOD89rCQIJ0p>.

REFERENCES

- [1] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex fourier series," *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965.
- [2] Xilinx, *Zynq-7000 SoC Technical Reference Manual*, Xilinx, uG585.
- [3] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Pearson, 2018.
- [4] A. V. Oppenheim and R. W. Schaffer, *Discrete-Time Signal Processing*, 3rd ed. Pearson, 2010.
- [5] K. K. Parhi, *VLSI Digital Signal Processing Systems: Design and Implementation*. Wiley, 1999.
- [6] *Fast Fourier Transform LogiCORE IP Product Guide*, AMD, 2025. [Online]. Available: <https://docs.amd.com/r/en-US/pg109-xfxt>
- [7] A. Mukherjee, A. Sinha, and D. Choudhury, "A novel architecture of area efficient fft algorithm for fpga implementation," 2015. [Online]. Available: <https://arxiv.org/abs/1502.07055>
- [8] A. Mukherjee and D. Choudhury, "An area efficient 2d fourier transform architecture for fpga implementation," 2018. [Online]. Available: <https://arxiv.org/abs/1810.06885>

- [9] AMD, *Vitis High-Level Synthesis User Guide*, AMD, uG1399.
- [10] R. Rajesh, S. J. Darak, A. Jain, S. Chandhok, and A. Sharma, "Hardware software co-design of statistical and deep learning frameworks for wideband sensing on zynq system on chip," arXiv preprint arXiv:2209.02661, 2022. [Online]. Available: <https://arxiv.org/abs/2209.02661>
- [11] J. Pu, S. Bell, X. Yang, J. Setter, S. Richardson, J. Ragan-Kelley, and M. Horowitz, "Programming heterogeneous systems from an image processing dsl," 2016. [Online]. Available: <https://arxiv.org/abs/1610.09405>
- [12] Xilinx, *Vivado Design Suite AXI Reference Guide*, Xilinx, uG1037.
- [13] —, *Vivado Design Suite User Guide: Design Analysis and Closure Techniques*, Xilinx, uG906.
- [14] —, *Vivado Design Suite User Guide: Power Analysis and Optimization*, Xilinx, uG907.